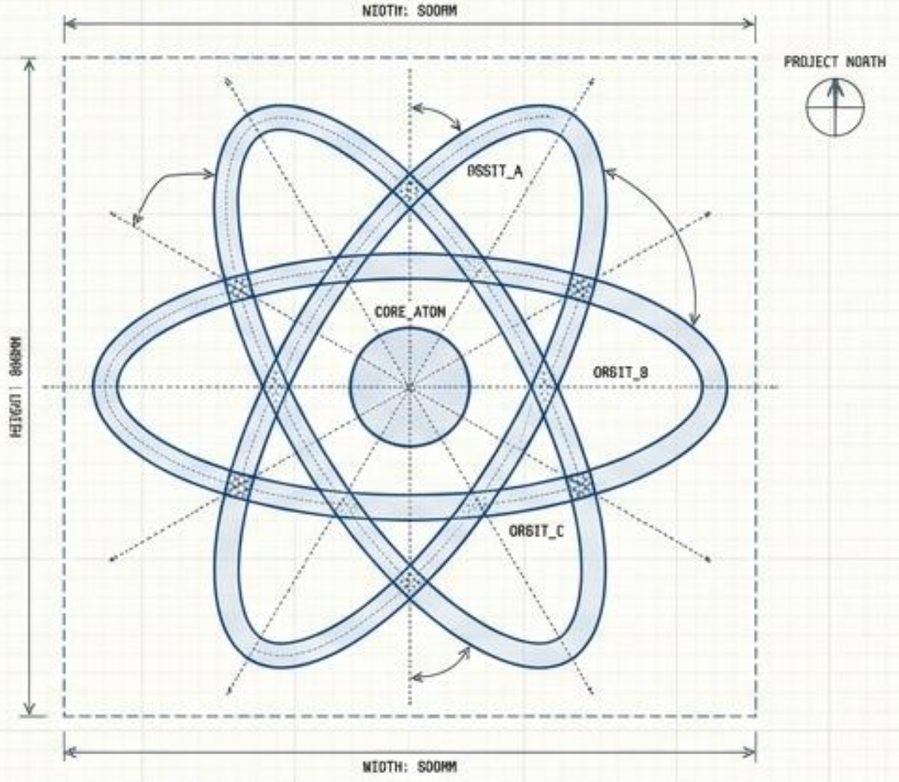
HitTastic! Architecture Blueprint

Component Design, Data Flow, and System Implementation (COM623)

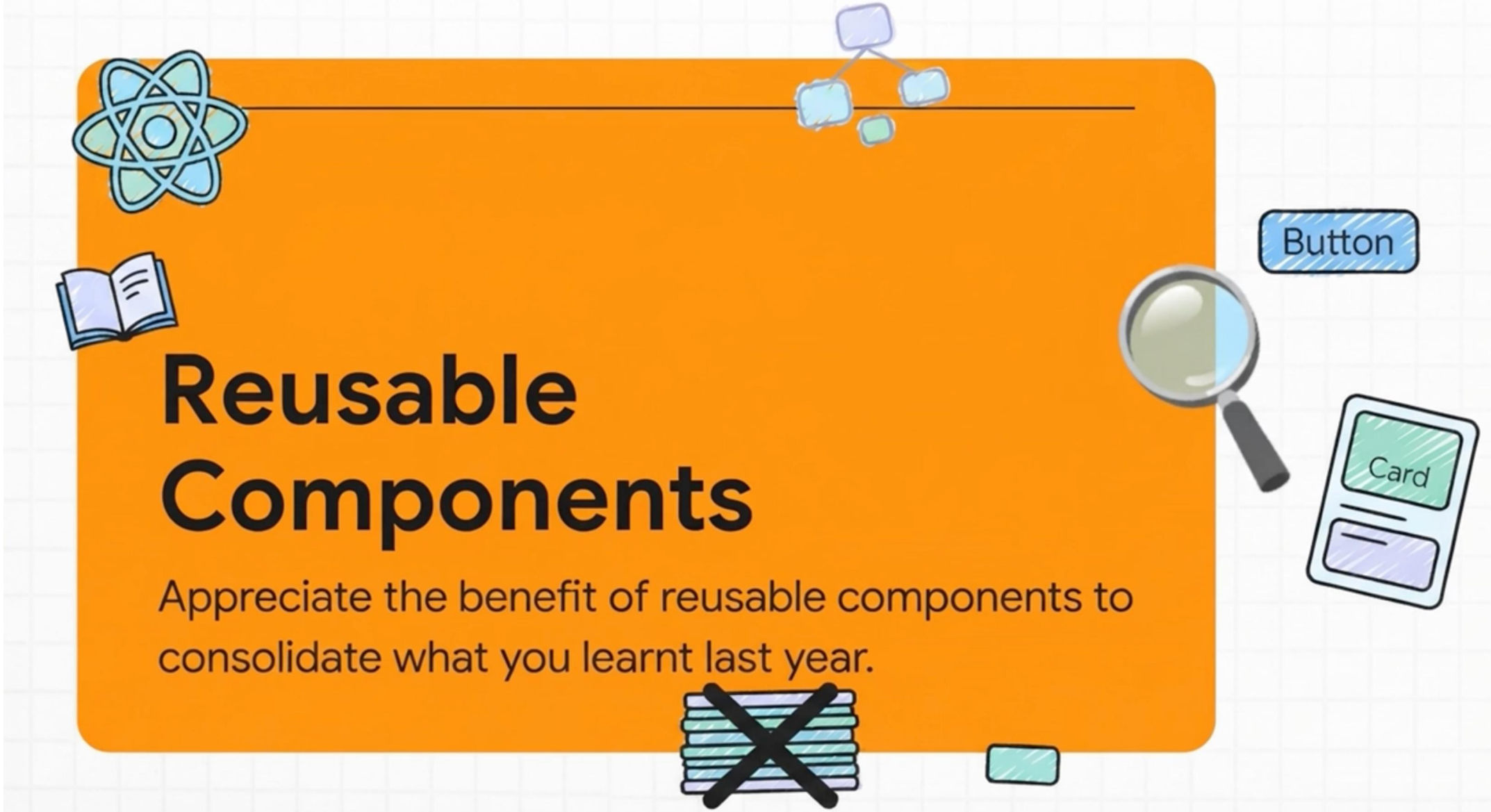


The React architectural blueprint for the HitTastic! application demonstrates the core component structure, unidirectional data flow, and state management system. The central graphic illustrates the fundamental 'atom' concept, emphasizing modularity, reusability, and structured state distribution across the application's technical layers.

DIRECTED LEARNING & LAB PRIMER

PROJECT: HitTastic! COM623	
DRAWING: ARCH-BP-001	REV: A
DATE: 2024-10-27	ENGINEER: J. DOE

Gemini Notebook



Reusable Components

Appreciate the benefit of reusable components to consolidate what you learnt last year.

Gemini Notebook

1

Welcome to HitTastic!

2

Designing the System

3

Component Breakdown

4

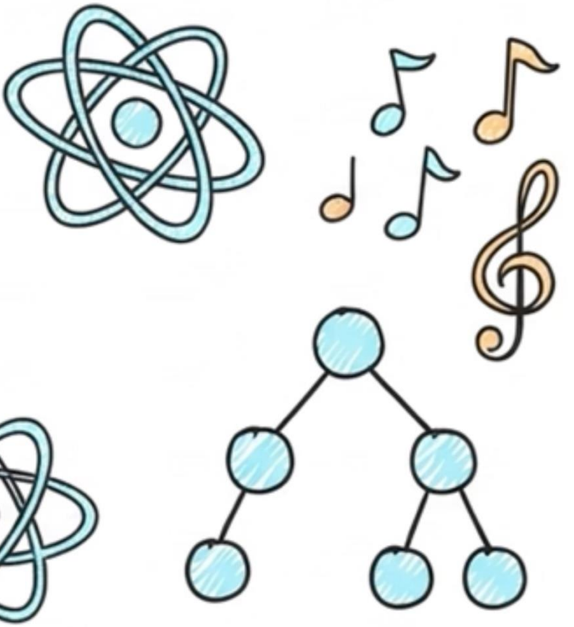
Coding the Application

Gemini Notebook

1

Welcome to HitTastic!

Consolidate React



Gemini Notebook



Gain practice to consolidate what you learnt and identify problems.

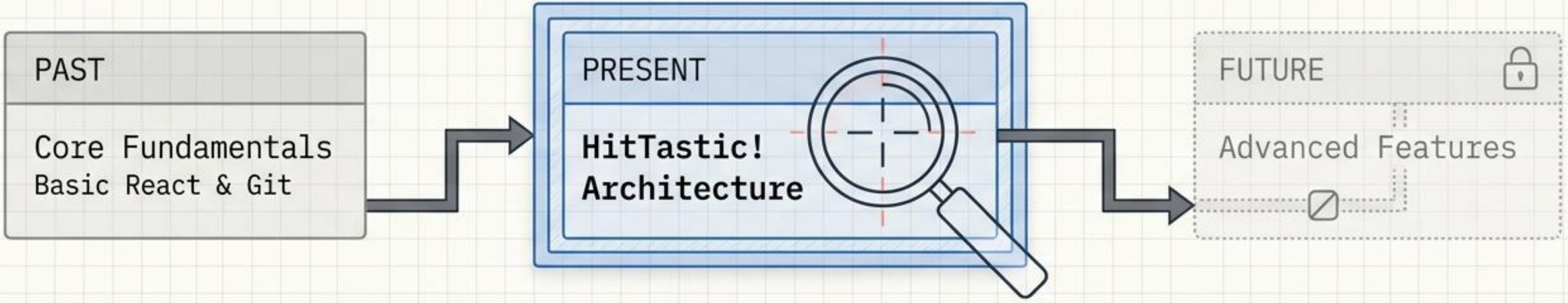


Introduce a range of new features to appreciate them fully.

Gemini Notebook

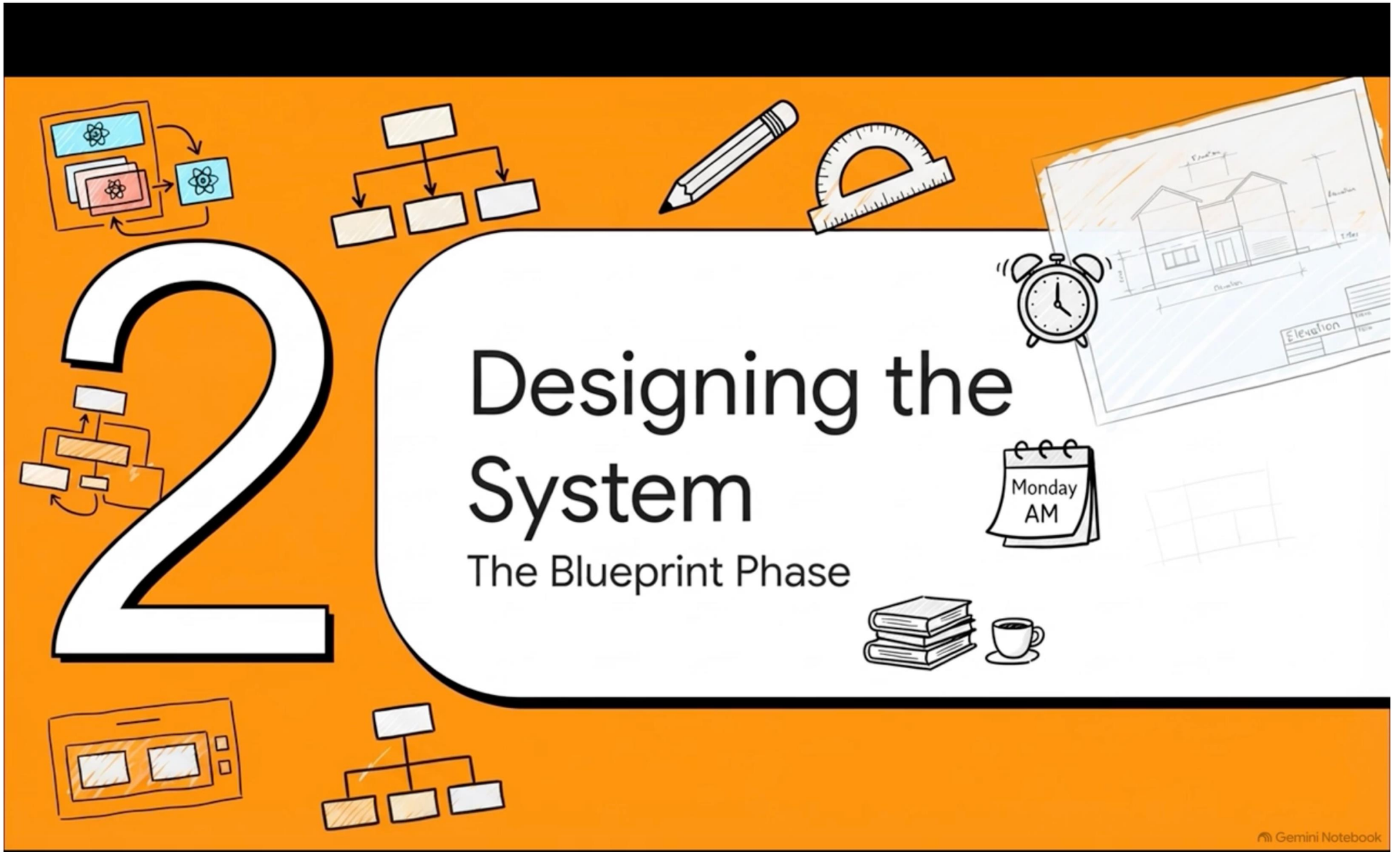
Consolidating the Foundation

Before unlocking advanced React features next week, we must solidify the core value of the framework: Reusable Components.



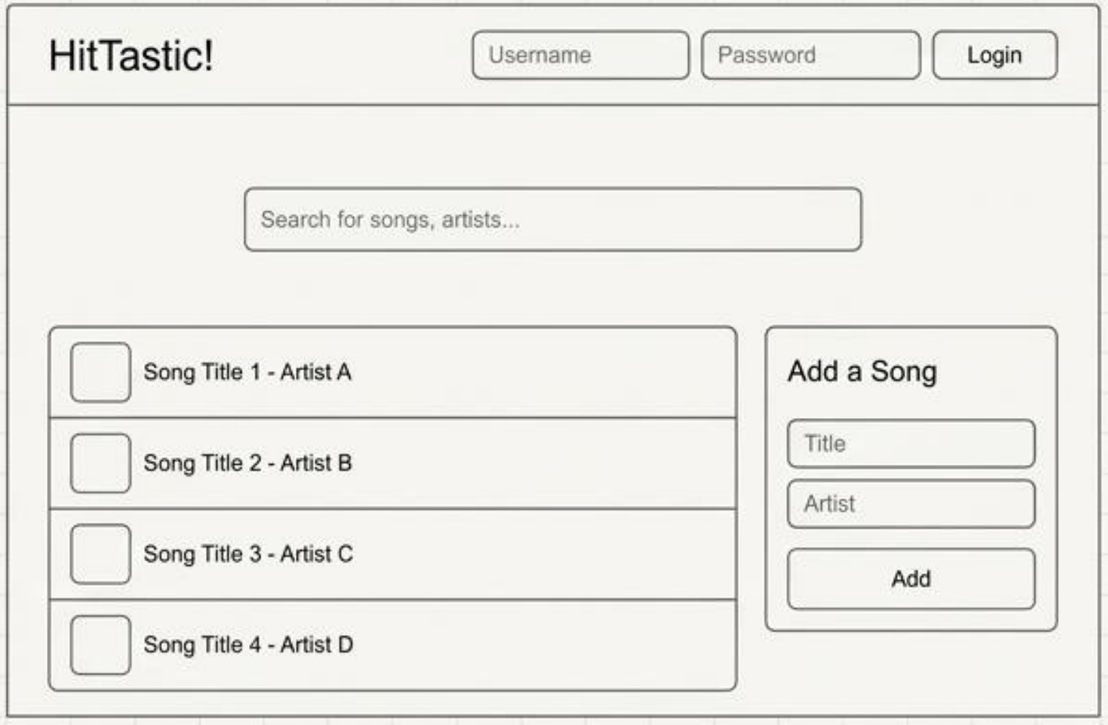
Strategic Intent: By designing the HitTastic! system today, you will intentioally the exact data-flow bottlenecks that next week's advanced features are built to solve.

Gemini Notebook

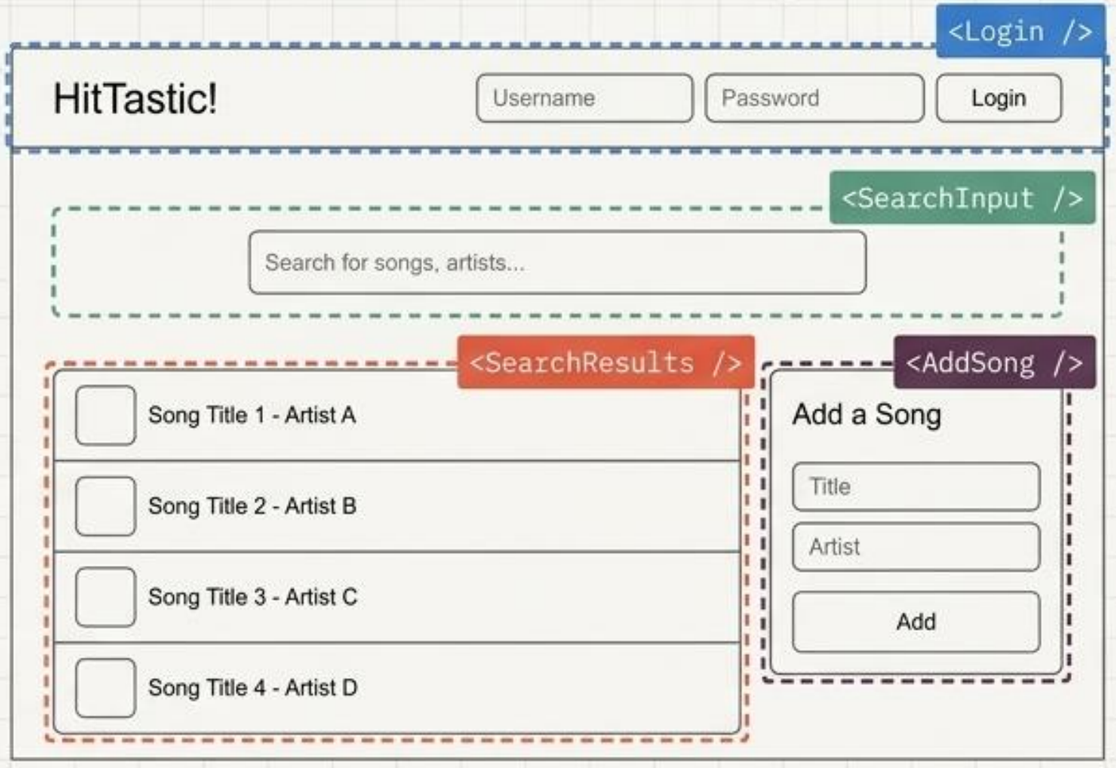

2
Designing the System
The Blueprint Phase
Monday AM
Elevation
Gemini Notebook

Seeing in Components

The Monolith (What the User Sees)



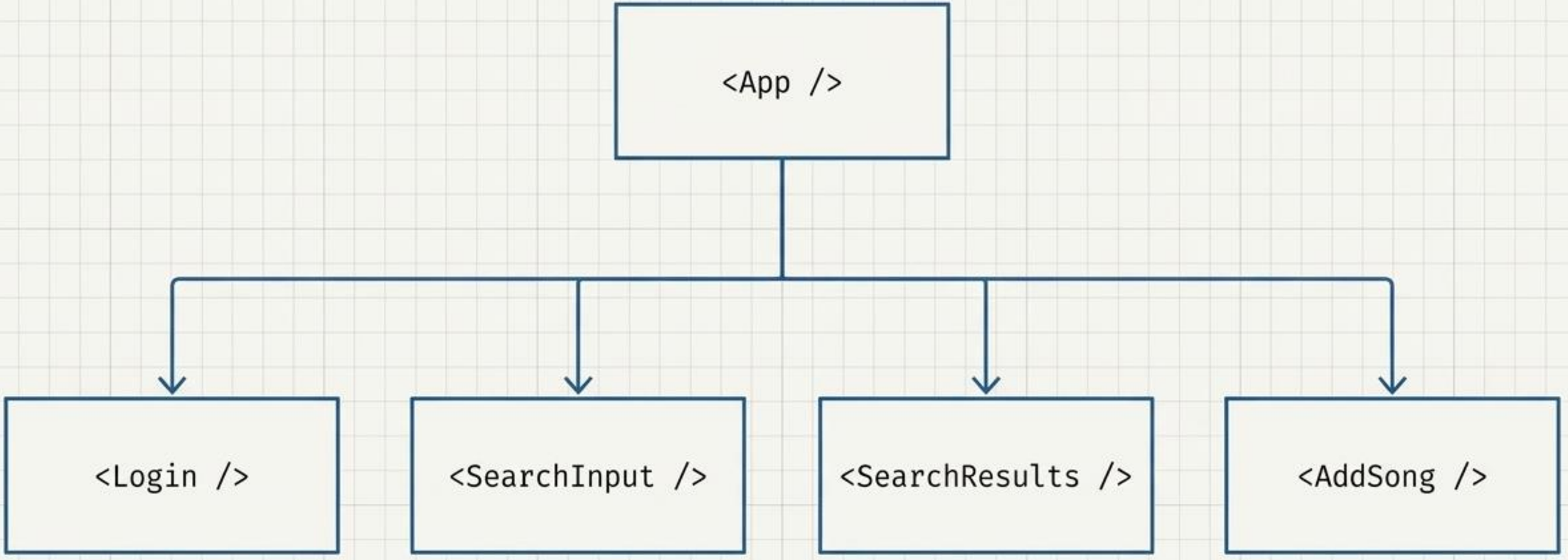
The Architecture (What the Engineer Sees)



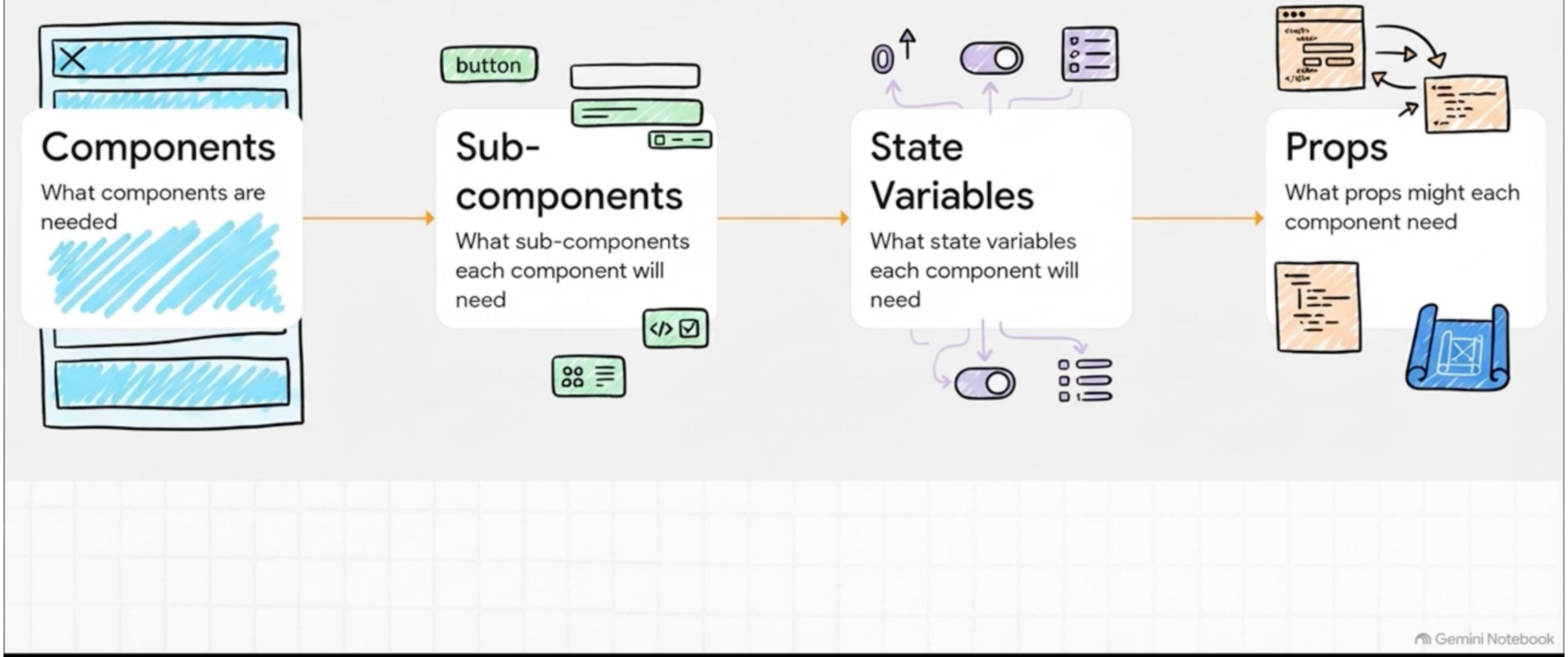
Gemini Notebook

The Component Hierarchy

Every React application is a tree. Identifying your components is only step one; defining their parent-child relationships dictates how your entire system will function and share data.

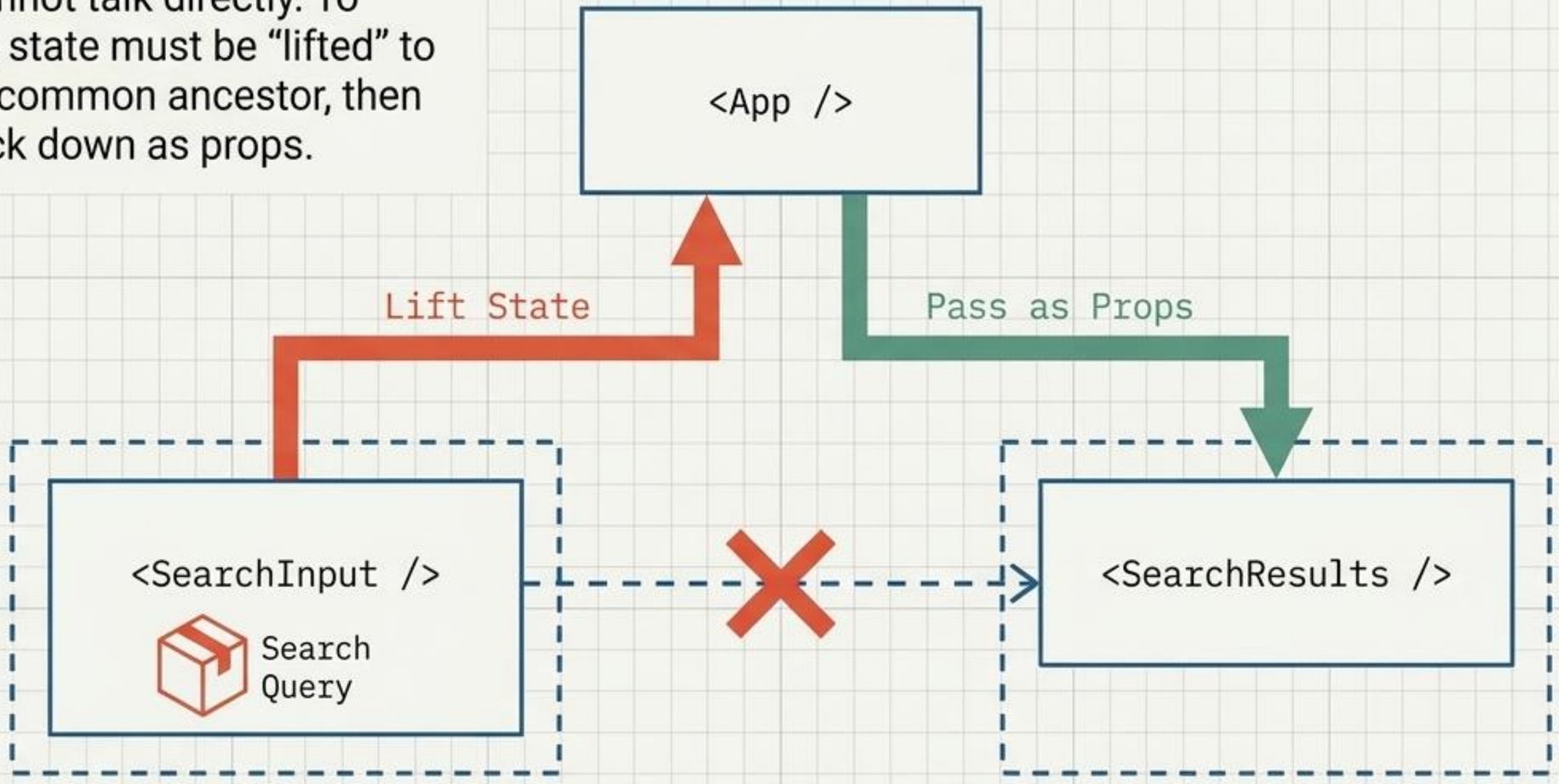


Gemini Notebook


Components
What components are needed
button
Sub-components
What sub-components each component will need
State Variables
What state variables each component will need
Props
What props might each component need
Gemini Notebook

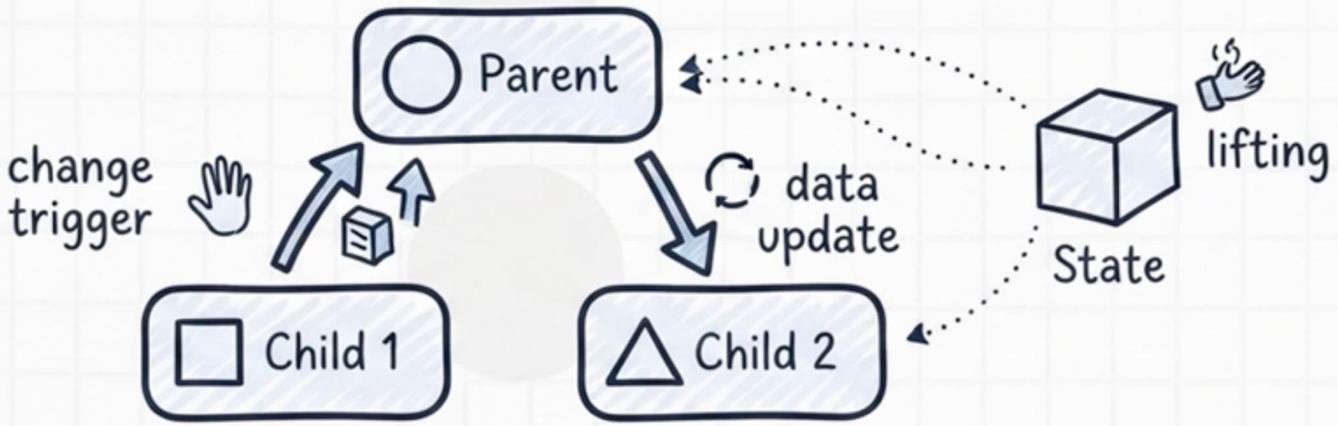
The Data Flow Rule: Lifting Up State

Siblings cannot talk directly. To share data, state must be “lifted” to the lowest common ancestor, then passed back down as props.



Gemini Notebook

What props might each component receive



Gemini Notebook

State & Props Distribution Matrix

Component	Required State (Variables)	Required Props (Inherited)
<App />	Global State (Search Query, User Auth)	None (It is the source)
<SearchInput />	Local input value	Function to update App's search query
<SearchResults />	None	Search Query from App to trigger fetch
<Login />	Username/Password fields	Function to update App's Auth state

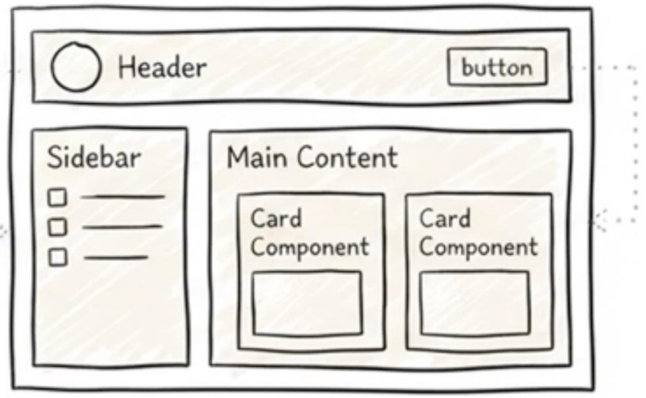
Note: Check your directed learning diagrams and logic against this architectural blueprint before coding.

Gemini Notebook

3

Component Breakdown

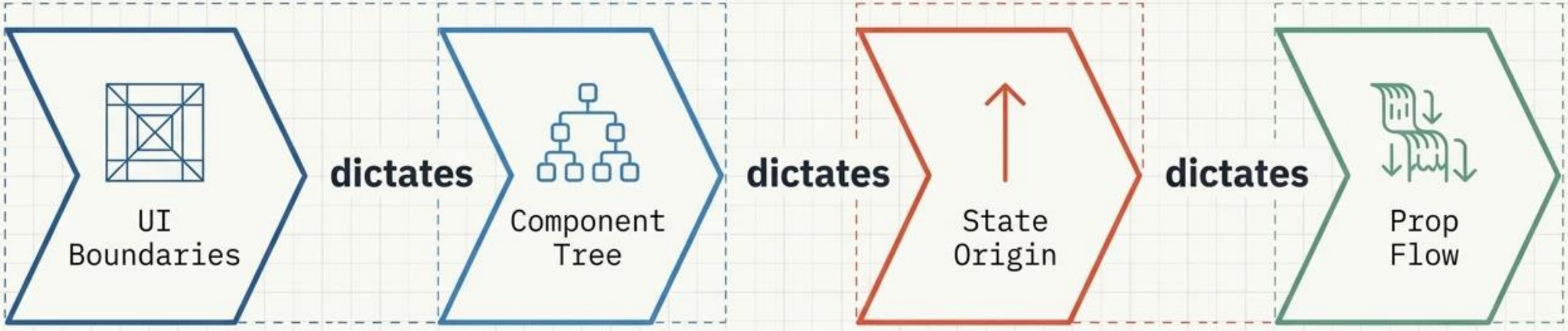
Mapping the UI



Gemini Notebook

The React Flow Framework

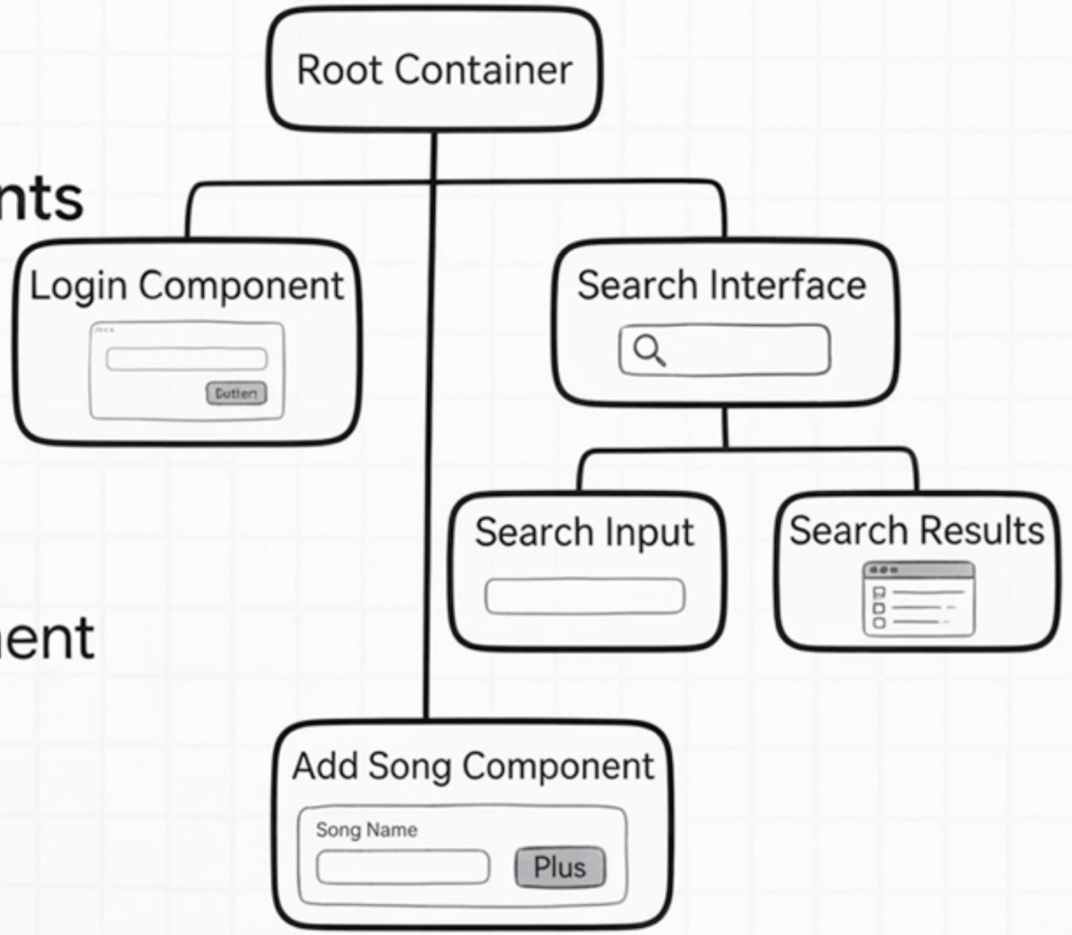
System design is a chain reaction. A mistake in drawing UI boundaries forces you to manage state in the wrong place, resulting in messy prop drilling. Design first, code second.



Gemini Notebook

Required Components

- Login component
- Search input
- Search results
- Add song component



Gemini Notebook



4

Coding the Application

Your Starter Kit







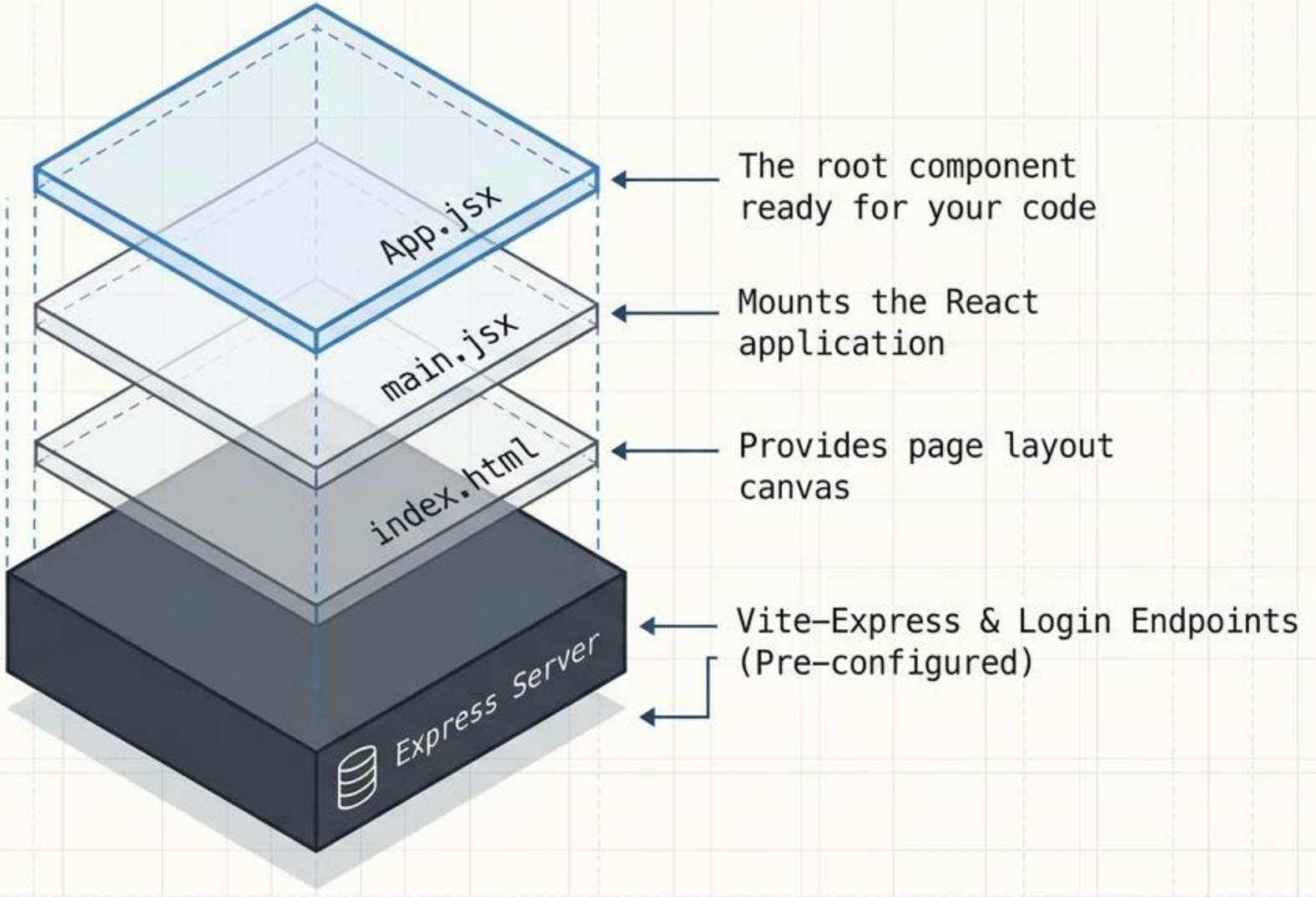




Gemini Notebook

Deconstructing the Starter Scaffold

The boilerplate is built so you can focus purely on React architecture. The backend is already wired to serve your frontend via Vite.



Gemini Notebook

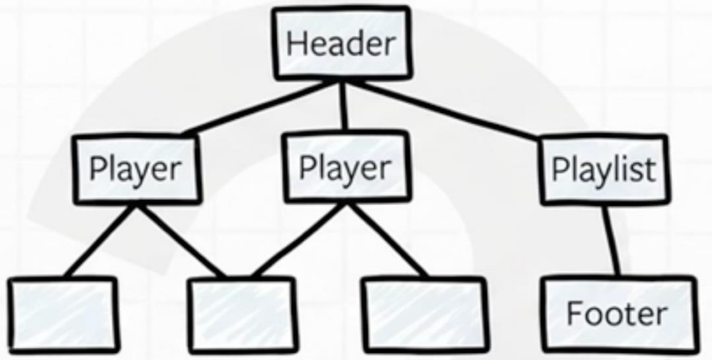
Execution: Coding Exercise 3.1

```
> git clone https://github.com/nwcourses/com623-wk3-starter.git
> cd com623-wk3-starter
> npm install
```

- [x] Clone the starter repository to your local environment.
- [x] Review `index.html` and `main.jsx` to understand the entry points.
- [x] Begin constructing the component tree inside `App.jsx` based on today's blueprint.

Gemini Notebook

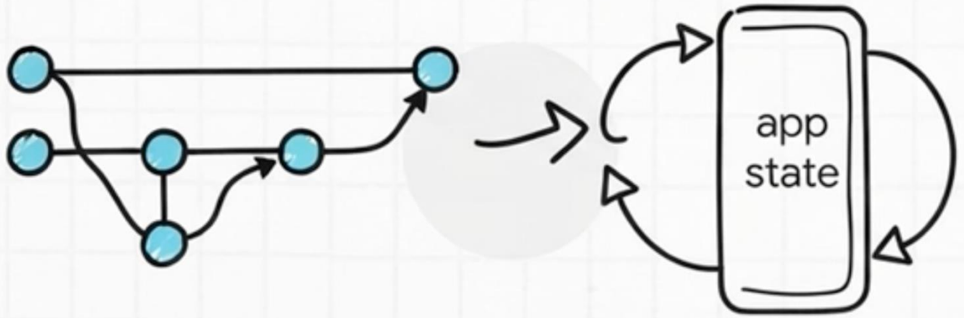






Are you ready to code HitTastic!?









Gemini Notebook